Model Context Protocol (MCP): Avaliação da Qualidade de Descrições de Tools com Base em Contexto de Código Fonte

Pedro H. P. Rodrigues[1], **Vinicius R. A. de A. Moreira**[1]

[1]Instituto de Ciências Exatas e Informática – Pontifícia Universidade Católica de Minas Gerais
Caixa Postal 500 – 30535-901 – Belo Horizonte – MG – Brazil

{vinicius,pedro}viniciusarantes133@gmail.com, pedrohenriquepr08@gmail.com

Resumo. *O Model Context Protocol (MCP) permite que agentes de Inteligência Artificial (IA) interajam com sistemas externos por meio de tools, cujas descrições em linguagem natural orientam sua utilização. Entretanto, descrições de baixa qualidade, caracterizadas pela presença de smells, podem comprometer a eficácia desses agentes. Nesse contexto, este trabalho realiza um estudo sobre a avaliação da qualidade dessas descrições por meio de uma rubrica multidimensional, da qual se derivam deficiências associadas a smells em ferramentas MCP, incorporando o código das ferramentas ao processo de avaliação. Para isso, são minerados repositórios de servidores MCP no GitHub e realizadas avaliações automatizadas das descrições das tools. O objetivo é comparar resultados obtidos a partir das descrições com aqueles enriquecidos pelo código da ferramenta. Espera-se demonstrar que a inclusão desse contexto técnico revela inconsistências entre descrição e implementação, tornando mais precisa a avaliação de qualidade e a identificação desses smells.*

Abstract. *The Model Context Protocol (MCP) enables Artificial Intelligence (AI) agents to interact with external systems through tools, whose natural language descriptions guide their usage. However, poor-quality descriptions, characterized by the presence of smells, can compromise the effectiveness of these agents. In this context, this work conducts a study on assessing the quality of these descriptions through a multidimensional rubric, from which deficiencies associated with smells in MCP tools are derived, incorporating the tools' source code into the evaluation process. To achieve this, MCP server repositories are mined from GitHub, and automated evaluations of tool descriptions are performed. The objective is to compare results obtained solely from descriptions with those enriched by the tool's code. It is expected to demonstrate that incorporating this technical context reveals inconsistencies between description and implementation, thereby improving both the quality assessment and the identification of these smells.*

Bacharelado em Engenharia de *Software* - PUC Minas
Trabalho de Conclusão de Curso (TCC)

Orientador de conteúdo (TCC I): João Pedro Batisteli - joaobatisteli@pucminas.br
Orientador acadêmico (TCC I): Leonardo Vilela - leonardocardoso@pucminas.br
Orientador do TCC II: Glender Brás - glendermedeiros@pucminas.br

Belo Horizonte, 28 de junho de 2026.

1. Introdução

O avanço dos modelos de linguagem (*Large Language Models*, LLMs) transformou o paradigma de interação entre humanos e máquinas [Hasan et al. 2026]. No entanto, a ausência inicial de padrões de comunicação entre LLMs e ferramentas externas gerou uma fragmentação técnica que motivou o surgimento do *Model Context Protocol* (MCP), este que é um protocolo aberto criado para padronizar a integração de modelos de IA com programas e fontes de dados externas. [Shi et al. 2026]. A adoção do protocolo tem crescido de forma acelerada, e um levantamento publicado no início de 2026 apontou que seus Kits de Desenvolvimento de Software (*Software Development Kit*, SDK) já registravam aproximadamente 97 milhões de *downloads* mensais [LLC 2026]. O principal diferencial do MCP reside no uso de ferramentas (*tools*) descritas em linguagem natural, que o agente pode selecionar e invocar em tempo de execução conforme a tarefa a ser resolvida. Esse funcionamento impõe, contudo, uma dependência crítica da qualidade documental, uma vez que o modelo não tem acesso ao código que implementa as ferramentas. As decisões de usá-las, como invocá-las e quais parâmetros fornecer são tomadas com base em suas descrições [Shi et al. 2026, Luo et al. 2025]. Dessa forma, assim como *code smells* comprometem a manutenibilidade de *softwares* tradicionais, descrições ambíguas ou incompletas comprometem o comportamento do agente, podendo induzir erros de planejamento, uso incorreto de parâmetros e alucinações [Mehedi Hasan et al. 2026].

Estudos recentes investigam a prevalência de anomalias (ou *smells*) em descrições de ferramentas MCP por meio de análise documental [Mehedi Hasan et al. 2026][Guo et al. 2025, Shi et al. 2026]. Essa abordagem avalia se a documentação comunica adequadamente o propósito, os parâmetros e as restrições de uso de uma ferramenta, examinando aspectos como clareza, completude e consistência, sem considerar a implementação técnica [Hasan et al. 2026]. Entretanto, evidências indicam que falhas relacionadas à configuração e à documentação representam 38,66% dos problemas observados em servidores MCP [Taraghi et al. 2026]. A elaboração e a manutenção dessas descrições dependem frequentemente de processos manuais desacoplados do código. Cabe destacar que, no MCP, a descrição não determina nem gera a implementação da ferramenta, mas atua como um artefato de documentação redigido manualmente para um código já existente, sendo este o que define o comportamento efetivo da ferramenta em execução. A prevalência de inconsistências entre descrição e implementação observada em servidores reais [Shi et al. 2026] evidencia que os dois artefatos podem divergir, o que motiva o uso do código como fonte adicional de evidência no processo de avaliação da qualidade da descrição. Por esse motivo, existe o risco de surgirem discrepâncias semânticas entre a documentação e a implementação real. Diante disso, existe uma lacuna na **análise da qualidade das descrições das**

ferramentas MCP, que, sem o contexto do código, pode não ser suficiente para a avaliação de sua qualidade.

A análise desse problema é relevante para a evolução sustentável de sistemas baseados em agentes de Inteligência Artificial (IA). Inconsistências semânticas podem reduzir a eficácia dos agentes e ampliar a superfície de exposição a vulnerabilidades, incluindo ataques de injeção de comandos [Taraghi et al. 2026]. Além disso, investigações recentes identificaram a presença de anomalias— como ambiguidade de propósito, omissão de parâmetros e ausência de limitações declaradas (Seção 2) — em grande parte das ferramentas disponíveis no ecossistema MCP [Hasan et al. 2026], e relatórios de segurança alertam que essas mesmas ambiguidades documentais podem favorecer a execução de ações não autorizadas ou a exposição indevida de dados sensíveis [National Security Agency 2026]. Nesse contexto, a incorporação de informações provenientes do código apresenta potencial para complementar abordagens tradicionais de avaliação de qualidade. Tal incorporação permite confrontar a descrição textual com a implementação efetiva da ferramenta, fornecendo evidências adicionais sobre possíveis inconsistências.

Tendo em vista essa lacuna, este trabalho tem como objetivo geral **investigar se a avaliação da qualidade de descrições de ferramentas MCP é influenciada pela disponibilização do código da ferramenta ao avaliador.** Para alcançar esse objetivo, definem-se os seguintes objetivos específicos: (i) Examinar se o fornecimento do código da *tool* resulta em maior precisão na avaliação da qualidade de sua descrição; e (ii) Identificar os atributos de avaliação mais afetados pela adição do código da *tool*; e (iii) Realizar uma análise estatística sobre os resultados obtidos nas avaliações.

Esses objetivos são orientados pelas seguintes questões de pesquisa: **RQ1**: o fornecimento do código da *tool* ao avaliador aumenta a precisão da avaliação da qualidade de sua descrição, em relação à avaliação baseada apenas na descrição? **RQ2**: quais atributos da rubrica de qualidade documental são mais afetados pela adição do código da *tool*? **RQ3**: as diferenças observadas entre os escores dos cenários com e sem código são estatisticamente significativas?

Como parte da investigação, será conduzida uma análise sobre um conjunto de servidores MCP de código aberto, selecionados a partir de critérios de popularidade e maturidade do projeto. A avaliação irá se concentrar no confronto entre as descrições em linguagem natural das ferramentas e sua lógica de implementação, representada por informações estruturais extraídas do código. Espera-se que essa abordagem possibilite a identificação de inconsistências que não são facilmente observáveis por métodos exclusivamente textuais, como parâmetros implementados, mas não documentados, ou funcionalidades descritas sem correspondência evidente na implementação. Dessa forma, busca-se produzir evidências empíricas capazes de apoiar ou refutar a hipótese de que a incorporação do contexto técnico pode ampliar a capacidade de detecção de *smells* em ferramentas MCP. Essa avaliação é operacionalizada por meio de uma rubrica de seis dimensões de qualidade documental, pontuadas em escala Likert de cinco pontos por modelos de linguagem atuando como avaliadores (*LLM-as-a-Judge*), dos quais se derivam as deficiências associadas aos *smells* investigados (Seção 4).

As próximas seções deste artigo estão organizadas da seguinte forma. A Seção 2

apresenta a Fundamentação Teórica sobre MCP, qualidade no contexto de engenharia de software e detecção de *smells*. A Seção 3 discute os trabalhos relacionados. Por fim, a Seção 4 descreve os materiais e métodos empregados na condução do estudo.

2. Fundamentação Teórica

Nesta seção, são descritos os principais tópicos que contextualizam e fundamentam a pesquisa. Inicialmente, é apresentado o conceito de qualidade de software. Em seguida, é apresentado o MCP, protocolo responsável por padronizar a comunicação entre agentes e ferramentas. Posteriormente, são abordadas as descrições em linguagem natural utilizadas por ferramentas MCP, elemento central para o processo de tomada de decisão dos agentes. Por fim, são discutidos os conceitos relacionados aos *smells* em descrições de ferramentas.

2.1. Qualidade de Software

Em sistemas baseados em agentes de Inteligência Artificial (IA), artefatos textuais como as descrições de ferramentas MCP orientam diretamente o comportamento dos modelos: é a partir desse texto — e não do código em si, ao qual o agente não tem acesso — que ele decide quando e como agir. Uma descrição de baixa qualidade constitui, portanto, um defeito de qualidade de software com efeito comportamental observável sobre o agente, e não apenas uma falha documental isolada.

A qualidade, no âmbito da Engenharia de Software, pode ser compreendida como o grau em que um produto de software atende aos requisitos especificados e às necessidades e expectativas de seus usuários e demais partes interessadas [ISO/IEC/IEEE 2017]. Segundo Pressman e Maxim (2016), a qualidade de software envolve tanto a conformidade com requisitos funcionais e de desempenho explicitamente declarados quanto a aderência a padrões de desenvolvimento e a características implícitas esperadas de todo software desenvolvido profissionalmente. Normas internacionais como a ISO/IEC 25010 [ISO/IEC 2011] estabelecem modelos de qualidade que decompõem esse conceito em características mensuráveis, tais como adequação funcional, confiabilidade, usabilidade e manutenibilidade, permitindo que a qualidade seja avaliada de forma sistemática e objetiva. A Essa avaliação, portanto, não se restringe ao código, estendendo-se também aos artefatos que compõem o software, como documentação, especificações e descrições, cuja clareza e precisão impactam diretamente a compreensão e o uso correto do sistema.

É nessa dimensão da qualidade de software — já prevista pela ISO/IEC 25010, mas cujo efeito se torna direto e mensurável quando quem consome a documentação é, ele próprio, um agente de IA que a interpreta automaticamente — que este trabalho se concentra: a qualidade da documentação das ferramentas MCP, tratada aqui como uma via de acesso à qualidade de software do sistema como um todo, e não como um fator à parte dela.

2.2. Model Context Protocol

O *Model Context Protocol* (MCP) é um padrão aberto que define como aplicações de IA se conectam a fontes de dados e ferramentas externas. Ele atua como uma camada de padronização entre o modelo e os sistemas que ele consome, de modo análogo ao papel que o USB-C desempenha ao uniformizar a conexão de periféricos de *hardware*

[Guo et al. 2025]. Tecnicamente, o protocolo adota uma arquitetura cliente-servidor baseada em mensagens JSON-RPC, na qual um *host* orquestra um ou mais servidores responsáveis por expor suas funcionalidades de maneira uniforme [Shi et al. 2026].

Cada servidor MCP disponibiliza suas capacidades por meio de três primitivas: as ferramentas (*tools*), que executam ações; os recursos (*resources*), que fornecem dados contextuais; e os *prompts*, que oferecem modelos de instrução [Guo et al. 2025]. Esse modelo padronizado dispensa a escrita de código de integração específico para cada plataforma e estabelece convenções comuns para a descoberta e a utilização de funcionalidades, o que favorece a interoperabilidade e o reúso de ferramentas no ecossistema [Hasan et al. 2026].

2.3. Descrições de Ferramentas

No MCP, as ferramentas dependem de descrições em linguagem natural como principal guia para os agentes baseados em LLMs. Diferentemente de sistemas tradicionais, que utilizam assinaturas de código estritas, a orquestração por IA exige que o propósito, os parâmetros e as restrições de cada funcionalidade sejam explicados textualmente, pois é a partir desse texto que o modelo decide quando e como invocar cada recurso [Mehedi Hasan et al. 2026].

A qualidade dessa interface linguística determina a capacidade do agente de selecionar as ferramentas corretas e de fornecer argumentos coerentes durante a execução. Uma descrição ambígua ou incompleta pode induzir o modelo a erros de planejamento, resultando em chamadas inválidas ou no uso de parâmetros inadequados [Shi et al. 2026]. A descrição deixa de ser, assim, uma documentação passiva e passa a funcionar como um artefato de especificação que influencia diretamente a tomada de decisão do sistema.

2.4. Smells em Descrições

O conceito de *smell* refere-se a características estruturais que sugerem problemas de qualidade em um artefato, ainda que não configurem defeitos funcionais, dificultando sua compreensão e manutenção [Mehedi Hasan et al. 2026]. Embora originalmente associado ao código, o conceito foi progressivamente estendido a artefatos em linguagem natural, como os *requirements smells*, que identificam ambiguidades e imprecisões em especificações de requisitos [Femmer et al. 2017], e os *documentation smells*, que apontam problemas de qualidade em documentação de APIs [Khan et al. 2021].

No âmbito do MCP, o termo aplica-se a ambiguidades, inconsistências e omissões nas descrições de ferramentas, como a ausência de especificação dos parâmetros de entrada e saída, do comportamento esperado e das limitações da ferramenta [Mehedi Hasan et al. 2026]. Por exemplo, uma ferramenta cuja descrição se limita a "processa os dados" configura um *smell* de vagueza: o texto não especifica quais dados são aceitos, qual transformação é aplicada nem qual saída é produzida, impedindo que o agente determine quando a ferramenta deve ser invocada. A detecção e a correção desses problemas tornam-se, portanto, condições relevantes para a confiabilidade do ecossistema, uma vez que descrições precisas sustentam o raciocínio dos agentes e favorecem uma execução de tarefas mais estável.

3. Trabalhos Relacionados

Esta seção apresenta estudos sobre a qualidade das descrições MCP e a identificação de *smells*. A literatura mapeia desafios de manutenção e vulnerabilidades que comprometem o desempenho de agentes autônomos. A análise desses trabalhos fundamenta a lacuna motivadora desta pesquisa. Busca-se superar a avaliação isolada de documentações por meio da integração do código, garantindo a fidelidade entre o texto e a implementação real.

Hasan et al. (2026) investigaram a qualidade das documentações de 856 ferramentas no ecossistema MCP por meio de uma análise empírica fundamentada em LLMs. O estudo aplicou uma rubrica de seis dimensões focada na clareza e completude do conteúdo textual. Os resultados revelaram que 97,1% das descrições possuem ao menos um *smell*, o que reduz a precisão dos agentes na seleção de parâmetros. O presente estudo parte dessa investigação e avança em uma direção não explorada pelos autores, sendo ela a incorporação do código ao processo de avaliação. Enquanto a abordagem anterior analisa as descrições de forma isolada, este trabalho utiliza o contexto técnico, extraído via grafos de chamadas, para confrontar a intenção declarada com a implementação real, verificando em que medida o acesso à lógica interna do servidor altera os escores obtidos nas dimensões da rubrica proposta pelos autores.

Em seu estudo sobre inconsistências entre descrições e implementações de ferramentas MCP, Shi et al. (2026) investigam a prevalência e os impactos de inconsistências semânticas entre descrições de ferramentas e seus respectivos códigos. O método consistiu em propor uma taxonomia de inconsistências descrição-código e um detector baseado em LLM para analisar pares de descrição e implementação em larga escala em servidores MCP reais. Os resultados revelaram que 9,93% das ferramentas analisadas apresentam inconsistências e que 35% dos servidores possuem ao menos um caso desse problema, evidenciando riscos relacionados à seleção incorreta de ferramentas, efeitos colaterais não declarados e falhas de segurança. Esse estudo sustenta a premissa desta pesquisa ao demonstrar que informações relevantes para compreender o comportamento de uma ferramenta frequentemente não estão presentes em sua descrição, o que fornece evidências de que a análise do código pode enriquecer a avaliação da qualidade de descrições MCP realizada por LLMs.

O panorama de adoção e maturidade do ecossistema MCP é mapeado empiricamente por Guo et al. (2025). O trabalho implementa um rastreador sistemático que analisa mais de 8.400 projetos em grandes repositórios para caracterizar o estado real da adoção do protocolo. Entre os achados, os autores constatam que mais da metade dos projetos listados são inválidos, abandonados ou de baixo valor agregado, o que evidencia a heterogeneidade de qualidade do ecossistema. Esse estudo fornece uma visão macro do MCP, mas concentra-se na caracterização dos repositórios e não examina a consistência entre as descrições das ferramentas e sua implementação no código. Este trabalho situa-se nesse ponto não coberto, ao investigar a qualidade das descrições no nível da ferramenta e o papel do código em sua avaliação. O dado de baixa maturidade reportado pelos autores também fundamenta o critério de popularidade adotado na seleção da amostra, descrito na Seção 4.

Com foco no desempenho dos agentes, Luo et al. (2025) propuseram o MCP-Universe, um *benchmark* para avaliar modelos de linguagem em servidores MCP reais.

O estudo emprega avaliadores baseados em execução, que validam o resultado prático das chamadas de ferramentas em vez de analisar apenas a semelhança textual das respostas. Ao avaliar mais de 20 modelos em 231 tarefas distribuídas por 11 servidores, os resultados revelaram que mesmo modelos de fronteira atingem taxas de sucesso abaixo de 45%, com falhas frequentemente associadas ao uso incorreto de interfaces de ferramentas pouco familiares. O presente trabalho investiga uma possível causa raiz dessas falhas, verificando se a divergência semântica entre a descrição textual de uma ferramenta e seu comportamento real é o fator que induz os agentes ao erro nesses cenários.

Em seu estudo sobre avaliação automática de modelos de linguagem, Zheng et al. (2023) investigam a viabilidade da abordagem *LLM-as-a-Judge*, na qual modelos de linguagem são utilizados para avaliar a qualidade das respostas produzidas por outros modelos. O método consistiu na comparação entre julgamentos realizados por GPT-4 e avaliações humanas em *benchmarks* de conversação. Os resultados mostraram que o GPT-4 alcançou concordância superior a 80% com avaliadores humanos, desempenho comparável ao observado entre os próprios especialistas. Este trabalho fundamenta metodologicamente a pesquisa proposta ao fornecer evidências da confiabilidade de avaliadores baseados em LLM, empregados neste estudo para analisar a qualidade de descrições MCP e o impacto da inclusão do código nesse processo de avaliação.

Em síntese, e demonstrado que a qualidade das descrições MCP é um fator crítico, pois discrepâncias entre o texto e a implementação comprometem a eficácia dos agentes. Embora estudos validem o uso de LLMs como avaliadores e a importância do código para a compreensão sistêmica, as abordagens atuais analisam as documentações de forma isolada. Diante dessa lacuna, este trabalho investiga como a incorporação do contexto técnico influencia a detecção de *smells* em descrições MCP.

4. Materiais e Métodos

Esta seção apresenta os procedimentos metodológicos adotados para a condução da pesquisa, bem como os instrumentos utilizados para coleta e análise dos dados. A pesquisa caracteriza-se como um estudo empírico, quantitativo e comparativo, cujo objetivo é investigar se a inclusão do código influencia a avaliação da qualidade de descrições de ferramentas MCP.

O fluxo metodológico adotado encontra-se resumido na Figura 1. O processo inicia-se com a coleta de repositórios MCP públicos, seguida pela extração das ferramentas e das informações sobre sua implementação. Em seguida, a qualidade das descrições das ferramentas é avaliada em dois cenários distintos: no primeiro, considerando a descrição em linguagem natural da ferramenta; no segundo, considerando a mesma descrição acompanhada pelo código correspondente. Nos dois cenários avalia-se a mesma descrição em linguagem natural; a única variável entre eles é a presença ou ausência do código correspondente, e não a linguagem em si. Por fim, os resultados obtidos são organizados e submetidos a uma análise comparativa e estatística.

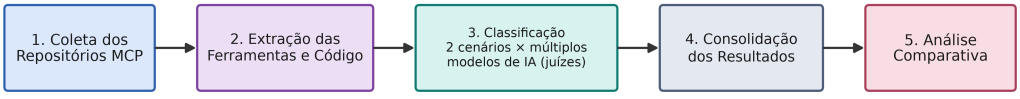


Figura 1. Fluxo metodológico adotado na pesquisa

4.1. Coleta dos Repositórios MCP

A primeira etapa consiste na construção do conjunto de dados que serve de base para os experimentos. Para isso, foram minerados repositórios públicos relacionados ao ecossistema MCP hospedados no GitHub. O objetivo dessa etapa é identificar implementações reais de servidores MCP que representem o cenário atual.

As consultas foram construídas a partir dos nomes dos pacotes necessários para a implementação do MCP. Para cada pacote, seu nome foi utilizado como termo de busca nos arquivos de dependências dos projetos. Essa etapa foi realizada por meio da busca de código da API REST do GitHub, que permite localizar repositórios com base em termos presentes no conteúdo de seus arquivos, ainda que a API limite cada consulta a um máximo de 1.000 resultados [Dong et al. 2023]. Dessa forma, foi necessário dividir a busca em consultas distintas, a fim de ampliar a quantidade de projetos identificáveis. Para isso, foi realizada uma consulta para cada combinação entre o nome da dependência e o respectivo arquivo de dependências em que ela poderia estar declarada. Como essa busca de código não devolve metadados do repositório em si, cada resultado foi complementado por uma consulta separada à API GraphQL do GitHub, de onde vieram o número de estrelas, a condição de *fork* e a linguagem principal usados nos filtros seguintes. Ao final, os repositórios duplicados foram unificados por identificador. Após isto, os que eram *fork* e os que não atingiam o piso mínimo de estrelas, ou cuja linguagem principal não estava entre as linguagens-alvo do estudo, foram removidos.

Para garantir a relevância e a maturidade dos *softwares* analisados, a coleta foi restrita a repositórios que possuam no mínimo 100 estrelas (*stars*). A adoção desse limiar fundamenta-se em estudos de mineração de repositórios que identificam o número de estrelas como uma medida direta de popularidade, sendo 100 estrelas um marco estatístico comum para filtrar projetos experimentais ou de baixo valor agregado, conhecidos como *toy repositories* [Borges et al. 2016]. Ao final, foram selecionados os 206 servidores mais populares encontrados e que possuíam ao menos uma tool, dobrando a amostra utilizada no estudo de Hasan et al. (2026) para ampliar a cobertura do ecossistema investigado.

A Figura 2 detalha o funcionamento interno dessa etapa. A mineração é realizada via API REST de busca de código do GitHub, segmentada por termos: cada combinação entre um sinal de dependência (por exemplo, `@modelcontextprotocol/sdk`) e um arquivo de manifesto onde esse sinal poderia estar declarado (por exemplo, `package.json`) gera uma sub-consulta independente; a linguagem não faz parte dessa segmentação e só é determinada depois, na hidratação de cada candidato. Cada repositório encontrado é então hidratado via GraphQL para a obtenção de seus metadados de linguagem principal e número de estrelas, informações não disponíveis diretamente na busca de código. Em seguida, os repositórios são unificados por identificador (removendo duplicatas encontradas por mais de uma sub-consulta) e submetidos à verificação de elegibilidade descrita acima. Os repositórios aprovados são ordenados por número de estrelas, e os mais relevantes são extraídos para compor a amostra final.

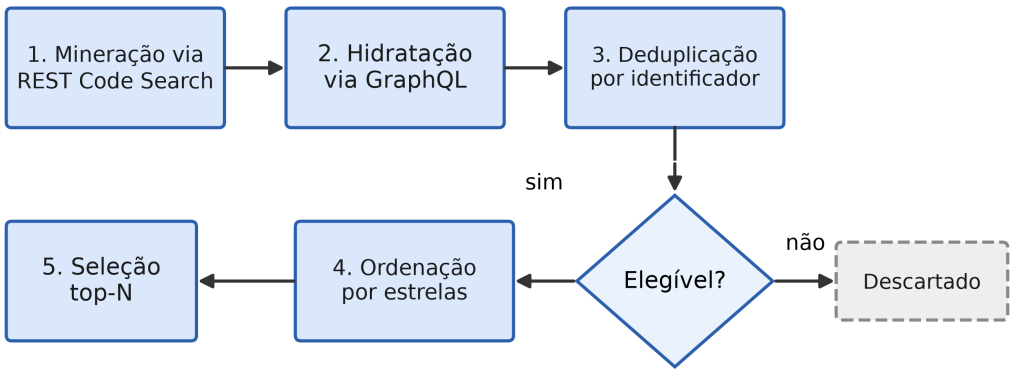


Figura 2. Funcionamento detalhado da Etapa 1 (Coleta dos Repositórios)

A busca por sinais de manifesto devolveu 12.058 candidatos brutos, todos provenientes de arquivos de dependência (`package.json`, `pyproject.toml`, `pom.xml` e equivalentes), únicos após a deduplicação por identificador. A Tabela 1 apresenta o funil de atrito aplicado sobre esse conjunto: o filtro de elegibilidade (repositório não ser *fork*, possuir pelo menos 100 estrelas e estar em uma das linguagens-alvo) reduziu esse conjunto para 723 repositórios.

Tabela 1. Funil de atrito da Etapa 1 (Coleta dos Repositórios MCP)

Etapa do filtro	Repositórios	% do bruto
Candidatos brutos (busca por manifesto)	12.058	100,0%
Aprovados no filtro (não-fork, estrelas $\geq$ mínimo, linguagem-alvo)	723	6,0%
Selecionados (qualificados pelo laço de *backfill*)	206	1,7%

Diferentemente de um corte simples pelas 206 posições mais bem colocadas, a seleção final passa por um laço de preenchimento (*backfill*): um repositório só permanece selecionado se, após a Etapa 2, confirmar pelo menos uma ferramenta; repositórios desqualificados são substituídos pelo próximo candidato do pool ainda não tentado, e o processo se repete até fechar as 206 vagas ou esgotar o pool. Entre os repositórios selecionados, a distribuição por linguagem principal está longe de ser uniforme, com forte concentração em Python e TypeScript (detalhamento completo no Apêndice B).

4.2. Extração das Ferramentas e do Código Relacionado

Após a obtenção dos repositórios, cada um foi clonado localmente para permitir a identificação automática das ferramentas disponibilizadas por cada servidor MCP, chegando ao resultado de 12.171 ferramentas. Para isso, foi desenvolvido um algoritmo de mineração capaz de percorrer a estrutura dos projetos e localizar as implementações associadas às ferramentas expostas pelo servidor. Para cada ferramenta identificada, foram extraídos sua descrição em linguagem natural e sua localização no código. Um repositório em que nenhuma ferramenta é localizada é tratado, na prática, como um falso positivo da etapa de coleta: ele é removido da amostra e substituído pelo próximo candidato ainda

não testado, repetindo-se o processo até completar o número de repositórios definido para o estudo ou esgotar os candidatos disponíveis, fechando o mesmo laço de *backfill* descrito na Seção 4.1.

A Figura 3 detalha o funcionamento interno dessa etapa (clonagem, detecção de ferramentas e construção do grafo de chamadas), incluindo o laço de *backfill* entre Etapas 1 e 2. Cada repositório selecionado é primeiro clonado e então indexado estaticamente: são catalogadas todas as definições de função/método e, por arquivo, os *imports* declarados. Em seguida, aplica-se o padrão estrutural do SDK correspondente à linguagem do repositório para localizar as ferramentas declaradas; se nenhuma ferramenta é encontrada, o repositório é desqualificado e substituído, reentrando no fluxo a partir da clonagem.

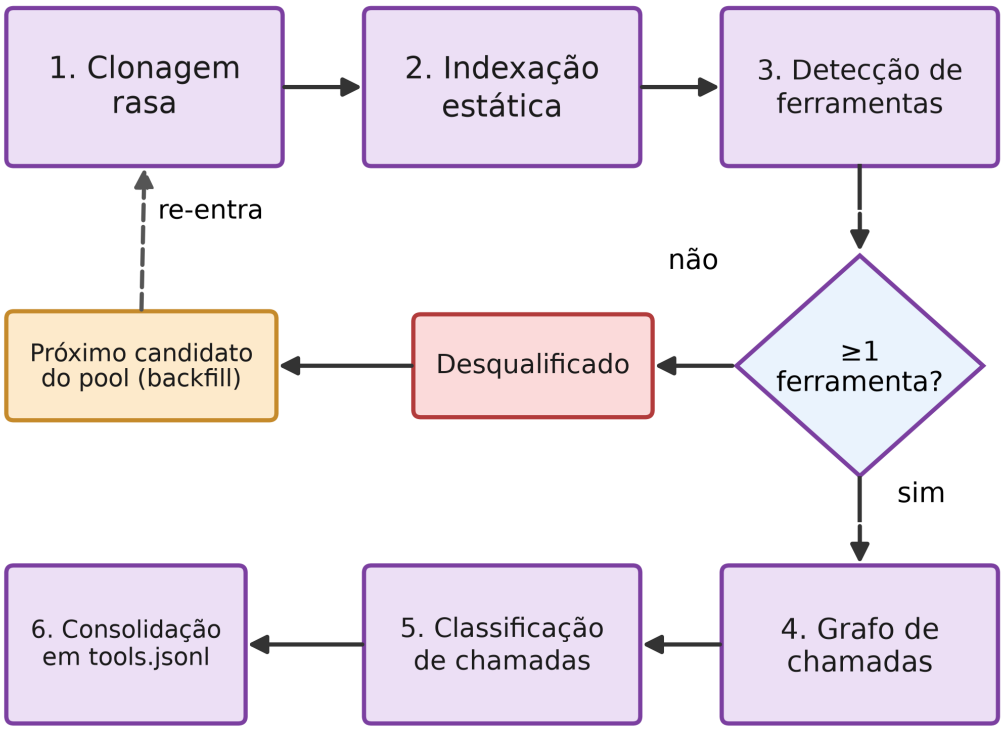


Figura 3. Funcionamento detalhado da Etapa 2 (Extração das Ferramentas)

Em seguida, foi realizada a extração do código relacionado a cada ferramenta. Como os servidores MCP possuem estruturas variadas, extrair apenas o método raiz de uma ferramenta pode não ser suficiente para capturar sua lógica, sendo necessário incluir também os métodos subsequentemente invocados. Para isso, foi construída uma árvore de chamadas (*call graph*) a partir do método inicial, expandida recursivamente até um limite máximo de três níveis de profundidade: o próprio método inicial constitui o primeiro nível; os métodos chamados diretamente por ele, quando resolvidos, formam o segundo nível; e as chamadas efetuadas a partir destes compõem o terceiro nível. A limitação da profundidade a três níveis é uma decisão de engenharia, não uma medição prévia de onde termina a lógica relevante de uma ferramenta: ela contém o custo da análise (e do texto eventualmente enviado ao avaliador) e elimina, por construção, qualquer risco de recursão

infinita, sem exigir detecção de ciclo separada (ver Apêndice A para o detalhamento completo da heurística de resolução). Um exemplo desse processo é apresentado na Figura 4.

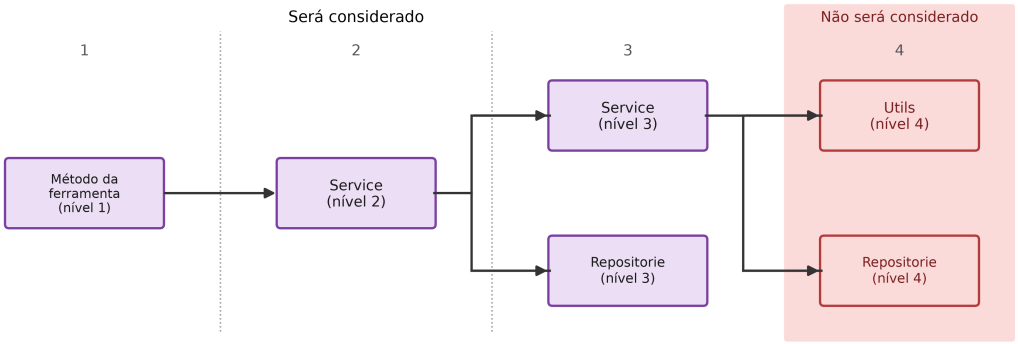


Figura 4. Exemplo de extração da árvore de chamadas de uma ferramenta

Ao final desta etapa, foi produzido um conjunto de dados estruturado contendo os servidores MCP analisados e as ferramentas identificadas em cada um deles. Para cada ferramenta, são armazenadas sua descrição em linguagem natural e o conjunto de métodos obtido a partir da árvore de chamadas. Esse conjunto de dados serve como entrada para as etapas subsequentes da pesquisa, nas quais as ferramentas dos repositórios coletados são avaliadas quanto à qualidade de suas descrições, com e sem o uso do código correspondente.

A Etapa 2 extraiu 12.171 ferramentas dos 206 repositórios selecionados; como a seleção final já incorpora o laço de *backfill*, todo repositório selecionado confirma, por construção, pelo menos uma ferramenta. A distribuição de ferramentas por linguagem segue de perto a distribuição de repositórios vista na coleta, com a mesma concentração em Python e TypeScript e uma assimetria adicional por repositório individual (detalhamento completo no Apêndice B).

4.3. Classificação das tools por diferentes modelos de IA

Na terceira etapa, utilizando as *tools* coletadas nas etapas anteriores, é conduzido um experimento comparativo estruturado em dois cenários de avaliação, executados em diferentes modelos de IA. No primeiro cenário, as descrições das ferramentas são avaliadas com base em seu conteúdo em linguagem natural. No segundo, essas mesmas descrições são avaliadas em conjunto com informações extraídas do código correspondente. Essa estratégia permite identificar o impacto da inclusão do contexto de implementação na avaliação da qualidade das descrições.

A Figura 5 detalha o funcionamento interno dessa etapa. Para cada ferramenta, são montados os dois cenários de avaliação descritos acima; cada cenário gera um *payload* contendo o nome da ferramenta, seu servidor de origem, a descrição e, quando aplicável, o código de contexto, enviado a cada juiz habilitado de forma concorrente. As respostas dos juízes podem resultar em uma avaliação concluída, em uma recusa de segurança do provedor, ou em um erro técnico durante o processamento. As duas primeiras são gravadas no arquivo de resultados e recebem uma entrada de *checkpoint*, permitindo retomar

a execução sem repetir avaliações já concluídas; o erro técnico não gera registro de resultado nem entrada de *checkpoint*, apenas um registro em um log de erros separado, e é reprocessado automaticamente na rodada seguinte.

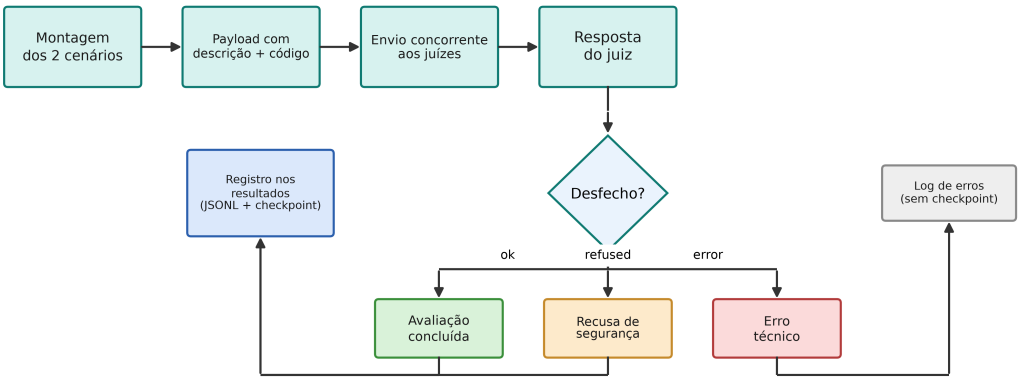


Figura 5. Funcionamento detalhado da Etapa 3 (Classificação das Ferramentas)

Em ambos os cenários, a avaliação é conduzida com base na rubrica proposta por Hasan et al. (2026), composta por seis componentes: *Purpose*, que avalia a clareza com que a descrição comunica a finalidade da ferramenta; *Guidelines*, que verifica se existem orientações sobre quando e como utilizá-la; *Limitations*, que analisa a explicitação de restrições, premissas ou casos de falha; *Parameter Explanation*, que mede a qualidade da documentação dos parâmetros de entrada; *Length and Completeness*, que avalia o nível de detalhamento e cobertura da descrição; e *Examples*, que verifica a presença de exemplos de utilização. Cada componente recebe uma pontuação em uma escala Likert de cinco pontos, sendo posteriormente utilizada para identificar deficiências associadas aos respectivos *smells* descritos pelos autores.

Em ambos os cenários, os modelos de linguagem atuam como o mesmo avaliador (*LLM-as-a-Judge*), recebendo as descrições das ferramentas e atribuindo as seis pontuações da rubrica acima. No primeiro cenário, essa avaliação é realizada sem qualquer acesso à implementação da funcionalidade analisada. No segundo cenário, o mesmo avaliador recebe adicionalmente o código correspondente, isto é, os trechos de código-fonte reunidos pela árvore de chamadas construída na Seção 4.2. Dessa forma, o avaliador pode confrontar a descrição fornecida com aspectos observáveis da implementação, verificando a existência de omissões, inconsistências ou informações potencialmente enganosas que não seriam perceptíveis apenas pela análise da descrição.

4.4. Consolidação dos Resultados por Cenário

Concluída a etapa de classificação, os resultados produzidos nos dois cenários experimentais são coletados e estruturados em uma base de dados unificada. Para cada ferramenta analisada, são armazenados identificadores que permitem sua associação ao servidor MCP de origem, bem como os resultados obtidos nos cenários com e sem a inclusão de informações provenientes do código, além de qual modelo de IA foi utilizado na avaliação.

A base de dados resultante contém os escores atribuídos pelo modelo para cada dimensão de qualidade avaliada, além das classificações finais produzidas em cada execução do experimento. Dessa forma, cada ferramenta possui dois conjuntos de resultados por modelo, diretamente vinculados à mesma descrição, diferindo apenas pela presença ou ausência das informações extraídas do código. Essa organização estabelece um relacionamento direto entre as avaliações realizadas sobre uma mesma ferramenta, preservando o pareamento necessário para as análises estatísticas subsequentes e possibilitando a comparação consistente dos resultados obtidos nos dois cenários experimentais.

A partir dessa base consolidada, é identificado um subconjunto adicional de ferramentas, denominado amostra de divergência unânime, utilizado como base para a análise qualitativa descrita na Seção 4.5. Uma ferramenta integra essa amostra quando, para pelo menos uma dimensão da rubrica, TODOS os juízes que a avaliaram atribuem notas diferentes entre o cenário sem código e o cenário com código, caracterizando um consenso de divergência entre os avaliadores. Para cada ferramenta selecionada, a amostra de divergência unânime reúne a justificativa (*reasoning*) de cada juiz nos dois cenários e o trecho de código correspondente, reconstruído pelo mesmo mecanismo descrito na Seção 4.2.

4.5. Análise Comparativa dos Experimentos

Na etapa final, os resultados obtidos são analisados com o objetivo de identificar diferenças entre os dois cenários de avaliação. Essa análise busca validar se a utilização do código de fato altera os resultados da avaliação das descrições, além de identificar quais atributos da rubrica utilizada para avaliar a qualidade são mais e menos impactados pela adição do código.

Inicialmente, realiza-se uma análise por meio do cálculo de medidas como média, mediana e desvio-padrão dos escores atribuídos a cada dimensão de qualidade avaliada, para identificar se há diferença numérica nos resultados gerados com e sem o contexto do código das ferramentas MCP, atendendo ao objetivo específico (ii) desta pesquisa. A partir dessa comparação, é possível avaliar quais atributos são mais afetados pela inserção do contexto técnico. Aplica-se também o Teste de Postos Sinalizados de Wilcoxon [Wilcoxon 1945], individualmente para cada dimensão de qualidade, a fim de verificar se as diferenças observadas entre as avaliações são estatisticamente significativas, concluindo o objetivo específico (iii). O teste é aplicado separadamente para cada modelo de linguagem utilizado, permitindo verificar se o efeito da inclusão do código se mantém consistente entre diferentes avaliadores. Como as duas avaliações são realizadas sobre a mesma ferramenta, o teste é adequado por considerar amostras pareadas e não exigir a suposição de normalidade dos dados. Os resultados dessa análise fornecem as evidências empíricas necessárias para apoiar ou refutar a hipótese de que a incorporação do contexto técnico amplia a capacidade de detecção de *smells* em descrições de ferramentas MCP.

Formalmente, para cada dimensão da rubrica e cada juiz, testa-se a hipótese nula de que a distribuição das diferenças pareadas entre os escores do cenário com código e do cenário sem código é simétrica em torno de zero, contra a hipótese alternativa de que a inclusão do código desloca essa distribuição. Como o teste é repetido para as seis dimensões da rubrica em cada juiz, avalia-se a necessidade de correção para comparações múltiplas antes de reportar significância. Pares de ferramentas em que os dois cenários

recebem exatamente o mesmo escore (diferença zero) são descartados do cálculo do teste, conforme o tratamento padrão de empates do Teste de Wilcoxon; a proporção de pares descartados por esse motivo é reportada junto com o resultado do teste, por dimensão, como indicador adicional do quanto a inclusão do código muda ou não a nota atribuída. Essa análise é implementada no mesmo script de análise dedicado da subseção anterior, a partir da base consolidada ali descrita.

Complementando a análise estatística acima, conduz-se uma análise qualitativa sobre a amostra de divergência unânime (Seção 4.4), com o objetivo de caracterizar não apenas quais dimensões da rubrica são mais afetadas pela inclusão do código, mas também que tipo de informação o código tende a revelar, e que a descrição, isoladamente, não comunica. Essa análise segue as fases de uma análise temática [Braun and Clarke 2006], complementadas por uma etapa final de análise de conteúdo dirigida [Hsieh and Shannon 2005], conforme ilustrado na Figura 6.

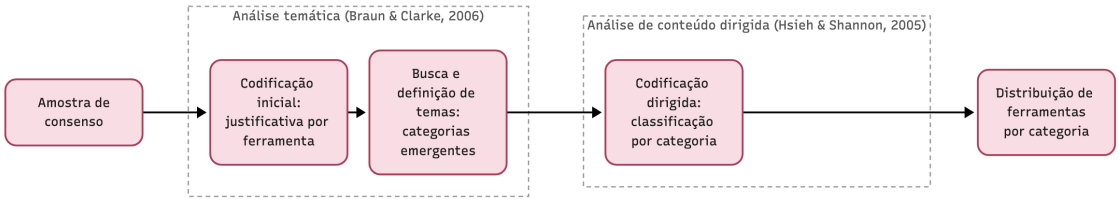


Figura 6. Fluxo detalhado da análise qualitativa sobre a amostra de divergência unânime

Nas duas fases iniciais da análise temática, familiarização com os dados e codificação inicial [Braun and Clarke 2006], para cada ferramenta da amostra de divergência unânime um modelo de linguagem (Gemini) é instruído a ler a justificativa (*reasoning*) de cada juiz nos dois cenários para o(s) componente(s) em que houve divergência, confrontá-la com o trecho de código correspondente, e sintetizar, em inglês, uma única justificativa concisa explicando o que o código revelou e que a descrição não comunicava: uma omissão de parâmetro, um comportamento ou formato de saída não documentado, uma limitação não divulgada, ou uma contradição direta entre descrição e implementação. O modelo é explicitamente instruído a não inferir uma conexão entre a mudança de nota e um achado do código quando a justificativa original do juiz não oferecer evidência concreta para isso, registrando essa ausência de evidência em vez de inventar uma explicação plausível; o texto completo do *prompt* utilizado encontra-se no Apêndice C.

Nas três fases seguintes, busca, revisão e definição dos temas [Braun and Clarke 2006], é realizada uma única chamada a um modelo de linguagem (DeepSeek), contendo o conjunto completo de justificativas produzidas na fase anterior, com o objetivo de identificar, de baixo para cima e sem partir de uma lista pré-definida, as categorias recorrentes que descrevem o mecanismo pelo qual o código revela uma inconsistência, por exemplo um parâmetro documentado como configurável mas na realidade fixado no código, ou uma estrutura de retorno mais rica do que a descrição sugere. O uso de um modelo distinto do Gemini nesta fase é uma decisão técnica, não metodológica: o volume de texto enviado numa única chamada excede o teto de taxa do tier gratuito do Gemini, enquanto o DeepSeek, já integrado ao projeto

como um dos juízes, não impõe esse limite. Cada categoria resultante é delimitada por uma descrição textual própria e exige estar fundamentada em ocorrências de mais de uma ferramenta, de modo a não introduzir uma categoria em função de um único caso idiossincrático (*prompt* completo no Apêndice C).

Concluída a análise temática, aplica-se uma análise de conteúdo dirigida [Hsieh and Shannon 2005]: o conjunto fixo de categorias produzido na fase anterior é utilizado para classificar cada ferramenta da amostra de divergência unânime, permitindo que uma mesma justificativa seja associada a mais de uma categoria, ou a nenhuma quando não houver correspondência, já que um mesmo achado de código pode evidenciar mais de um tipo de omissão simultaneamente. Diferentemente da fase de definição dos temas, na qual o conjunto completo de justificativas é analisado de uma só vez para permitir que os padrões emerjam por comparação entre casos, essa classificação é realizada individualmente, uma ferramenta por vez: ao se solicitar a um modelo de linguagem que classifique milhares de itens em uma única resposta, o desempenho do modelo tende a degradar-se para os itens posicionados no meio da lista [Liu et al. 2024], risco que a classificação item a item elimina por construção (*prompt* completo no Apêndice C).

A classificação dirigida produzida na etapa anterior é, por sua vez, submetida a uma verificação manual sobre uma amostra de ferramentas, cujo tamanho é determinado pela Fórmula de Cochran para populações de tamanho conhecido [Chaokromthong et al. 2021] (Equação 1), que fornece o número mínimo de casos a observar sem prejudicar a validade do estudo. De forma semelhante ao estudo de Hasan et al. (2026) , utiliza-se uma taxa de 95% de nível de confiança (Z = 1,96), uma proporção de população estimada de 0,5 (p) e 10% de margem de erro (ε), o que leva ao resultado de n = 96, arredondado para 100 ferramentas.

$$n = \frac{\dfrac{Z^2 p(1-p)}{\varepsilon^2}}{1 + \left(\dfrac{Z^2 p(1-p)}{\varepsilon^2 N} \right)} \tag{1}$$

A seleção dessas 100 ferramentas é estratificada por categoria de mecanismo, com vagas distribuídas o mais igualmente possível entre as 16 categorias (mais a classificação `none`) em vez de proporcionalmente à frequência de cada uma, de modo que a amostra não fique concentrada na categoria mais comum da população. Dentro de cada categoria, a seleção prioriza ferramentas de repositórios e de componentes divergentes da rubrica de Hasan et al. (2026) ainda não representados na amostra, para também não concentrar em poucos repositórios ou sempre no mesmo componente. Para cada ferramenta selecionada, compara-se a categoria atribuída pelo modelo na etapa anterior com uma leitura independente da justificativa sintetizada na primeira etapa e da justificativa (*reasoning*) original de cada juiz para o componente da rubrica em que houve divergência, registrando-se concordância ou discordância.

Como limitação metodológica, declara-se que as três etapas de geração de justificativa, identificação de categorias e classificação dirigida são conduzidas por um único *"codificador"* automatizado, sem um processo paralelo de codificação humana nem cálculo de concordância entre avaliadores (*inter-rater reliability*) nessas três eta-

pas. A verificação manual descrita acima mitiga essa limitação apenas para a etapa de classificação, confrontando o resultado do modelo com uma leitura independente; as etapas de geração de justificativa e de identificação de categorias permanecem sem essa validação cruzada.

5. Resultados

Esta seção apresenta os resultados da avaliação por rubrica descrita na Seção 4, obtidos até o momento da redação deste artigo com o juiz DeepSeek-V4.1-Flash: 8.648 avaliações concluídas, das quais 4.304 ferramentas foram avaliadas nos dois cenários experimentais (*description_only* e *with_source*), permitindo o pareamento necessário para o Teste de Wilcoxon. A avaliação pelos demais juízes configurados no pipeline (Gemini 3.5 Flash-Lite e Qwen2.5-14B-Instruct, Seção 4) está em execução; a comparação entre avaliadores distintos é tratada como direção de trabalho futuro.

5.1. Comparação entre Cenários e Significância Estatística

A Tabela 2 apresenta, para o juiz DeepSeek-V4.1-Flash, o escore médio de cada componente da rubrica nos dois cenários (apenas descrição *versus* descrição com código), a diferença entre eles (Δ) e o resultado do Teste de Postos Sinalizados de Wilcoxon, com correção de Benjamini-Hochberg para comparações múltiplas entre as seis dimensões. A proporção de empates (pares em que os dois cenários recebem exatamente o mesmo escore) varia por dimensão, de 61,8% a 94,6%.

Tabela 2. Comparação entre cenários por componente da rubrica: DeepSeek-V4.1-Flash (n pareado = 4.304)

Componente	Só descrição	Desc.+código	Δ	p (BH)	Signif.
Purpose	3,43	3,30	-0,13	<0,001	Sim
Guidelines	1,64	1,62	-0,03	<0,001	Sim
Limitations	1,33	1,30	-0,03	<0,001	Sim
Parameter Explanation	2,03	1,97	-0,06	<0,001	Sim
Length and Completeness	1,81	1,80	-0,02	0,044	Sim
Examples	1,20	1,20	-0,00	0,425	Não

A inclusão do código reduz o escore médio nas seis dimensões da rubrica, com diferença estatisticamente significativa em cinco delas (todas exceto *Examples*); a maior queda ocorre em *Purpose* ($\Delta = -0,13$), seguida por *Parameter Explanation* ($\Delta = -0,06$). Esse padrão é compatível com a hipótese de que o código expõe lacunas da descrição não percebidas quando apenas o texto é avaliado: ao confrontar a descrição com uma implementação mais completa do que o texto sugere, o avaliador atribui uma pontuação mais conservadora. A generalização desse resultado para outros juízes depende da conclusão da coleta de avaliações e é tratada como direção de trabalho futuro.

5.2. RQ1: O fornecimento do código da *tool* ao avaliador aumenta a precisão da avaliação da qualidade de sua descrição, em relação à avaliação baseada apenas na descrição?

O aumento ou a diminuição da nota atribuída a uma *tool* não indica, por si só, uma melhora ou piora na precisão da avaliação da descrição: indica apenas que houve uma divergência

na nota após a introdução do código, podendo essa divergência refletir tanto uma melhora real de precisão quanto viés ou ruído introduzido pelo próprio avaliador. Responder à RQ1 exige, portanto, identificar o **mecanismo** pelo qual o código altera a nota, não apenas a direção da mudança. Saber que a nota mudou no componente *Guidelines*, por exemplo, já está implícito no próprio dado coletado (campo `componente`), e não revela se o código tornou a avaliação mais precisa.

Essa identificação foi conduzida pela análise qualitativa descrita na Seção 4.5, sobre a amostra de divergência unânime (Seção 4.4): 2.685 ferramentas em que todos os juízes que as avaliaram concordam que a nota mudou, no mesmo componente, entre *description_only* e *with_source*. A análise temática indutiva sobre o conjunto completo de justificativas dessas ferramentas produziu as 16 categorias de mecanismo apresentadas na Tabela 3, cada uma descrevendo um tipo específico de informação que o código revela e que a descrição, isoladamente, não comunica.

Tabela 3. Categorias de mecanismo identificadas pela análise temática indutiva sobre a amostra de divergência unânime

Categoria	Descrição
Superfície de Entrada Não Documentada	A assinatura ou o esquema de entrada do código aceita parâmetros, filtros, identificadores de sessão ou combinações obrigatórias que a descrição nunca menciona.
Parâmetros Fantasmas ou Divergentes	A descrição documenta parâmetros que o código não possui, ou que foram renomeados, removidos ou substituídos na implementação.
Valores Fixos e Padrões Divergentes	Um valor apresentado como configurável está, na realidade, fixado no código, ou o valor padrão documentado diverge do valor efetivamente usado.
Limites Operacionais, Padrões e Paginação Não Divulgados	Tetos, truncamentos, *timeouts*, paginação e valores padrão não declarados são impostos pela implementação sem menção na descrição.
Modos de Falha e Erros Não Divulgados	Tratamentos de erro, exceções, falhas de dependência e mensagens de estado vazio da implementação não são antecipados pela descrição.
Dependências e Pré-requisitos de Ambiente Ocultos	Serviços externos, variáveis de ambiente, credenciais, binários ou configurações exigidos pelo código não são mencionados na descrição.
Controles de Acesso, Papel e Autorização	Permissões, autenticação, verificações de papel/administrador e outras restrições de acesso aplicadas no código estão ausentes da descrição.
Efeitos Colaterais Não Divulgados	O código produz efeitos colaterais (exclusões em cascata, escrita de arquivos, notificações) além da ação anunciada pela descrição.
Funcionalidade Anunciada Não Implementada	O código expõe um *stub*, um retorno fixo, ou simplesmente não implementa a funcionalidade que a descrição promete.
Sistema-Alvo Divergente	O código revela uma plataforma, formato de arquivo ou granularidade de recurso diferente da que a descrição declara.
Estrutura Concreta da Saída	A estrutura exata do retorno (campos, aninhamento, formatação, variações por sucesso/falha) é mais específica do que a descrição sugere.
Regras de Validação e Contratos de Formato Não Divulgados	Conjuntos de valores permitidos, formatos, regras de exclusão mútua ou campos obrigatórios fixados no código não são declarados na descrição.
Escopo de Processamento e Mecânica Interna Subestimados	A implementação realiza bem mais processamento ou mecânica interna (múltiplas sub-chamadas, *cache*, chamadas de rede, lógica assíncrona) do que a descrição sugere.
Semântica de Correspondência e Normalização de Entrada	O código processa entradas de forma não óbvia (correspondência por substring, normalização de apelidos, resolução difusa de nomes) sem que a descrição mencione essa semântica.
Contexto de Fluxo e Ativação	O código revela pré-requisitos de estado de sessão, ordem de chamadas entre ferramentas irmãs, ou contexto ambiente que a descrição não articula.
Confirmação de Encapsulamento Trivial	O código é um encaminhamento mínimo cujo comportamento já correspondia exatamente ao texto, sem revelar nada novo.

A classificação dirigida foi aplicada às 2.685 ferramentas da amostra de divergência unânime contra as 16 categorias da Tabela 3, permitindo múltiplas categorias por ferramenta. Apenas 3,9% das ferramentas (106 de 2.685) não casaram com nenhuma

categoria (`none`), indicando que o conjunto de categorias cobre a grande maioria dos mecanismos presentes na amostra. A Tabela 4 apresenta a frequência de cada categoria.

Tabela 4. Frequência de cada categoria de mecanismo na amostra de divergência unânime (n = 2.685; multi-*label*, a soma excede 100%)

Categoria	Ocorrências	%
Superfície de Entrada Não Documentada	1.492	55,6%
Modos de Falha e Erros Não Divulgados	700	26,1%
Estrutura Concreta da Saída	517	19,3%
Regras de Validação e Contratos de Formato Não Divulgados	500	18,6%
Limites Operacionais, Padrões e Paginação Não Divulgados	465	17,3%
Contexto de Fluxo e Ativação	359	13,4%
Escopo de Processamento e Mecânica Interna Subestimados	350	13,0%
Confirmação de Encapsulamento Trivial	245	9,1%
Valores Fixos e Padrões Divergentes	228	8,5%
Dependências e Pré-requisitos de Ambiente Ocultos	205	7,6%
Controles de Acesso, Papel e Autorização	143	5,3%
Semântica de Correspondência e Normalização de Entrada	127	4,7%
Parâmetros Fantasmas ou Divergentes	87	3,2%
Funcionalidade Anunciada Não Implementada	83	3,1%
Efeitos Colaterais Não Divulgados	77	2,9%
Sistema-Alvo Divergente	65	2,4%
(nenhuma categoria)	106	3,9%

Superfície de Entrada Não Documentada domina a amostra, presente em mais da metade das ferramentas (55,6%): o achado mais comum é que o código aceita parâmetros, filtros ou opções que a descrição simplesmente não menciona. Modos de Falha e Erros Não Divulgados (26,1%) e Estrutura Concreta da Saída (19,3%) seguem como o segundo e o terceiro mecanismos mais recorrentes. Mesmo as categorias menos frequentes mostram-se padrões reais, não casos isolados: Sistema-Alvo Divergente e Confirmação de Encapsulamento Trivial, as duas categorias menos frequentes da Tabela 4, ainda assim aparecem em 65 e 245 ferramentas, respectivamente.

A verificação manual confirmou a categoria atribuída pelo modelo em 72 das 100 ferramentas da amostra (72,0%). Nas 28 ferramentas restantes, a leitura independente da justificativa não sustentou a categoria atribuída; a maior parte desses casos concentra-se em confusões entre categorias de mecanismo com fronteiras próximas, por exemplo entre Contexto de Fluxo e Ativação e Superfície de Entrada Não Documentada, ou entre Parâmetros Fantasmas ou Divergentes e Valores Fixos e Padrões Divergentes, em vez de atribuições sem nenhuma relação com o conteúdo real da justificativa.

5.3. RQ2 Quais atributos da rubrica de qualidade documental são mais afetados pela adição do código da *tool*?

Ao comparar os atributos avaliados nos cenários com e sem acesso ao código, observa-se que a inclusão do código no método de avaliação impacta cada atributo de maneira distinta. A Figura 7 apresenta a proporção de casos em que a análise com acesso ao código resultou em melhora, piora ou manutenção da pontuação atribuída à descrição da *tool* para cada modelo de IA utilizado.

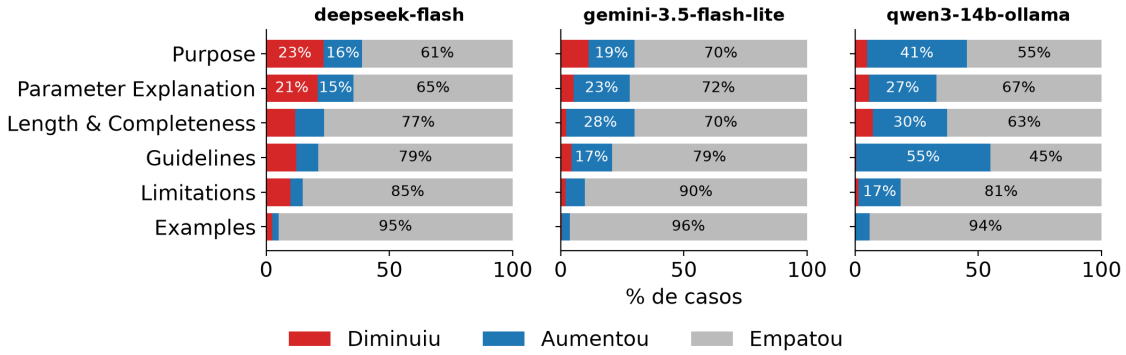


Figura 7. Distribuição proporcional da alteração nas pontuações por atributo.

Entre os modelos utilizados é possível perceber que o Deepseek teve, entre os três, uma maior tendência em diminuir a nota na avaliação com código em comparação a avaliação sem código da descrição das tools. Em contrapartida os modelos Gemini e Qwen3 tiveram uma tendência em aumentar a nota na avaliação com o código, sendo os resultados do Qwen3 o mais discrepante entre os três, o que evidencia uma menor concordância dele com os outros dois modelos utilizados.

Em vista dos atributos analisados, *Limitations* e *Examples* são os que apresentam menor variação de nota entre as avaliações com e sem código para todos os três modelos utilizados. *Purpose* é a que, em média, apresenta a maior variação de notas. Já o atributo de *Guidelines* possui uma grande discrepância da sua avaliação feita pelo Qwen3 em relação aos outros, sendo o terceiro com menos variação no Deppseek e Gemini, e sendo o único com menor numero de empates que aumentos de nota entre todas as avaliações pelo Qwen3. Para analisar o comportamento e a distribuição das pontuações em cada cenário, a Figura 8 apresenta uma série de *boxplots* comparando as avaliações com e sem acesso ao código para cada um dos atributos considerados em cada modelo.

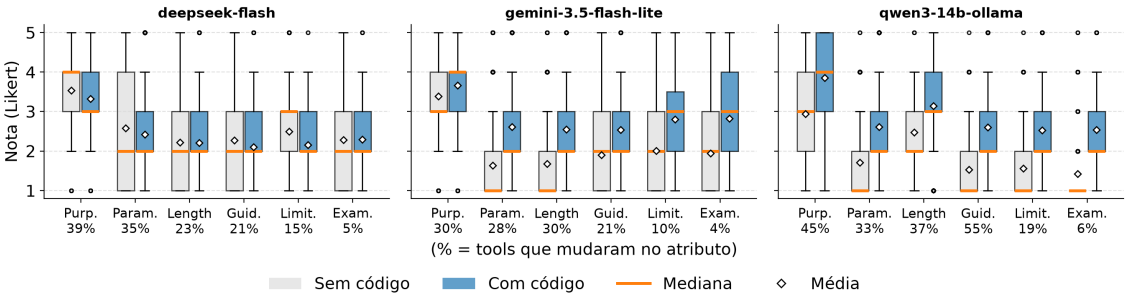


Figura 8. Distribuição das notas nos cenários com e sem código para os casos com alteração de pontuação. (média entre modelos)

Na construção dos *boxplots* da Figura 8, foram consideradas apenas as avaliações que apresentaram diferença de nota. A escolha dessa abordagem ocorreu porque a maioria das avaliações resultou em empate, o que ocultava a real diferença das avaliações com e sem código. Ao analisar apenas as descrições com notas diferentes, observa-se para o modelo do Deepseek uma tendência maior em diminuir a nota nas avaliações com

código, isso pode ser afirmado pela diminuição da mediana nos atributos de *Purpose* e *Limitations*, e pela diminuição da média em todos os atributos com exceção de *Length & Completeness* e *Examples*. Por outro lado os modelos utilizados do Gemini e do Qwen3 obtiveram a tendência oposta de aumentar a nota das avaliações do cenário com código em comparação ao cenário sem código, tendo a média e a mediana elevadas em todos os atributos, sendo os resultados do Qwen3 mais extremos em comparação ao dos outros dois modelos de IA.

5.4. RQ3 as diferenças observadas entre os escores dos cenários com e sem código são estatisticamente significativas?

Para responder à RQ3, foram empregados dois métodos de verificação da significância estatística das diferenças entre as duas medições: o Teste de Wilcoxon para Amostras Pareadas, cujos resultados estão apresentados na Tabela 5, e a análise de transição de quartis, detalhada na Tabela 6. Para a aplicação do Teste de Wilcoxon, adotou-se o nível de significância de 5% ($\alpha = 0{,}05$). Para a análise de transição de quartis, utilizaram-se como referência os quartis obtidos no cenário sem código, comparando-se a posição de cada par *tool*-atributo entre a avaliação sem código e a avaliação com código. Em ambos os métodos, utilizou-se a média das notas atribuídas pelos modelos de inteligência artificial analisados neste estudo.

Tabela 5. Resultados do Teste de Postos Sinalizados de Wilcoxon comparando as notas com e sem código, por atributo (Média dos modelos; pareado por tool).

Atributo	n diferentes	W	p-valor
Purpose	6130	5031832,0	$9,84 \times 10^{-222}$***
Parameter Explanation	5626	4526255,5	$1,34 \times 10^{-171}$***
Length & Completeness	5380	2145338,0	$< 10^{-300}$***
Guidelines	5956	1973603,0	$< 10^{-300}$***
Limitations	2430	667277,0	$2,30 \times 10^{-123}$***
Examples	902	67412,5	$3,64 \times 10^{-69}$***

*** $p < 0.001$; ** $p < 0.01$; * $p < 0.05$

A Tabela 5 apresenta, para cada um dos seis atributos, o número de ferramentas com avaliações diferentes entre os cenários (n), a estatística do teste de Wilcoxon (W) e o respectivo valor-p. Como é possível observar, as avaliações de todos os seis atributos apresentam diferenças estatisticamente significativas entre os cenários analisados ($p < 0{,}05$).

Tabela 6. Transição de quartil das notas (com vs. sem código), por atributo (Média dos modelos). Faixas de quartil definidas a partir da distribuição sem código (pareado por tool).

Atributo	Manteve o quartil	Subiu de quartil	Desceu de quartil
Purpose	66,2%	24,7%	9,2%
Parameter Explanation	62,1%	29,5%	8,4%
Length & Completeness	64,2%	30,2%	5,7%
Guidelines	60,5%	38,2%	1,3%
Limitations	89,0%	9,3%	1,8%
Examples	95,8%	3,6%	0,6%

Ao analisar os dados da Tabela 6 observa-se que os atributos que registraram maior proporção de mudança de quartil foram: *Purpose*, *Parameter Explanation*, *Length & Completeness* e *Guidelines*, com 33,9%, 37,9%, 35,9% e 39,5% dos casos totais alterados, respectivamente. Em contrapartida, os demais atributos apresentaram menor variação na distribuição por quartis, com os atributos *Limitations* e *Examples* registrando 11,1% e 4,2% de alteração entre as avaliações, o que concorda com os Ws registrados na Tabela 5 que apresentava valores mais baixos justamente para esses dois parâmetros.

Referências

Borges, H., Hora, A., and Valente, M. T. (2016). Predicting the popularity of github repositories. In *Proceedings of the 12th International Conference on Predictive Models and Data Analytics in Software Engineering*, pages 1–10.

Braun, V. and Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2):77–101.

Chaokromthong, K., Sintao, N., et al. (2021). Sample size estimation using yamane and cochran and krejcie and morgan and green formulas and cohen statistical power analysis by g* power and comparisions. *Apheit international journal of interdisciplinary social sciences and technology*, 10(2):76–86.

Dong, Y., Li, Z., Tian, Y., Sun, C., Godfrey, M. W., and Nagappan, M. (2023). Bash in the wild: Language usage, code smells, and bugs. *ACM Trans. Softw. Eng. Methodol.*, 32(1).

Femmer, H., Méndez Fernández, D., Wagner, S., and Eder, S. (2017). Rapid quality assurance with requirements smells. *Journal of Systems and Software*, 123:190–213.

Guo, H., Hao, Y., Zhang, Y., Xu, M., Lv, P., Chen, J., and Cheng, X. (2025). A Measurement Study of Model Context Protocol Ecosystem. *arXiv e-prints*.

Hasan, M. M., Li, H., Fallahzadeh, E., Rajbahadur, G. K., Adams, B., and Hassan, A. E. (2026). Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers. *ACM Trans. Softw. Eng. Methodol.* Just Accepted.

Hsieh, H.-F. and Shannon, S. E. (2005). Three approaches to qualitative content analysis. *Qualitative Health Research*, 15(9):1277–1288.

ISO/IEC (2011). Iso/iec 25010:2011 – systems and software engineering – systems and software quality requirements and evaluation (square) – system and software quality models. Technical report, International Organization for Standardization, Geneva, Switzerland.

ISO/IEC/IEEE (2017). Iso/iec/ieee 24765:2017 – systems and software engineering – vocabulary. Technical report, International Organization for Standardization, Geneva, Switzerland.

Khan, J. Y., Khondaker, M. T. I., Uddin, G., and Iqbal, A. (2021). Automatic detection of five api documentation smells: Practitioners' perspectives. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 318–329.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

LLC, A. (2026). The model context protocol: A status report for enterprise customer experience leaders. White paper, Avaya.

Luo, Z., Shen, Z., Yang, W., Zhao, Z., Jwalapuram, P., Saha, A., Sahoo, D., Savarese, S., Xiong, C., and Li, J. (2025). MCP-Universe: Benchmarking Large Language Models with Real-World Model Context Protocol Servers. *arXiv e-prints*.

Mehedi Hasan, M., Li, H., Krishnan Rajbahadur, G., Adams, B., and Hassan, A. E. (2026). Model Context Protocol (MCP) Tool Descriptions Are Smelly! Towards Improving AI Agent Efficiency with Augmented MCP Tool Descriptions. *arXiv e-prints*.

National Security Agency (2026). Model context protocol (mcp): Security design considerations for ai-driven automation. Cybersecurity report, NSA Cybersecurity Information.

Pressman, R. S. and Maxim, B. R. (2016). *Engenharia de Software: Uma Abordagem Profissional*. AMGH, Porto Alegre, 8 edition.

Shi, Y., Zhang, X., Zhang, X., Shen, X., Ouyang, H., Qiu, H., Zhang, M., and Yang, M. (2026). Description-code inconsistency in real-world mcp servers: Measurement, detection, and security implications. *arXiv preprint arXiv:2606.04769*.

Taraghi, M., Morovati, M. M., and Khomh, F. (2026). Real faults in model context protocol (mcp) software: a comprehensive taxonomy. *arXiv preprint arXiv:2603.05637*.

Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J., and Stoica, I. (2023). Judging llm-as-a-judge with mt-bench and chatbot arena. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S., editors, *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623. Curran Associates, Inc.

Apêndice

A. Algoritmo de Construção do Grafo de Chamadas e Composição do Contexto de Código

Este apêndice detalha o algoritmo desenvolvido para a construção do grafo de chamadas (*call graph*) de cada ferramenta MCP e para a composição do trecho de código usado como contexto adicional no cenário de avaliação que inclui código (Seção 4). O algoritmo é heurístico e baseado em análise estática via tree-sitter: a partir da função que implementa a própria ferramenta, resolve por nome as chamadas feitas dentro dela, até três níveis de profundidade, distinguindo chamadas resolvidas dentro do próprio repositório de chamadas externas ou dinamicamente determinadas, conforme ilustrado na Figura 4.

Procedimento de Extração

O algoritmo opera em três fases distintas, executadas uma vez por repositório:

1. **Indexação estática do repositório:** são construídos, a partir de todo o código do repositório, um índice de definições (nome simples e nome qualificado, `Classe.método`, de toda função/método encontrado, com referência ao seu corpo) e, por arquivo, um índice de *imports* (alias local usado no arquivo → módulo e nome original de onde veio). Ambos são heurísticas de escopo léxico, não uma resolução completa de símbolos.
2. **Construção recursiva do grafo de chamadas:** a árvore de cada ferramenta é construída a partir da função que a implementa (nível 1). Para cada chamada encontrada no corpo de uma função, uma heurística de resolução por nome é aplicada em ordem, valendo a primeira que casar: (i) receptor `self`/`this` → método da própria classe; (ii) mesmo nome definido no mesmo arquivo → função auxiliar local; (iii) receptor casa com um alias de *import* conhecido → busca no módulo importado; (iv) nome único em todo o repositório → resolve diretamente; (v) não encontrado em lugar nenhum (biblioteca externa, *stdlib*, ou chamada genuinamente dinâmica) → tratado como folha externa, sem recursão. Uma chamada resolvida vira um novo nó, expandido recursivamente até o nível 3; nós de nível 3 continuam sendo chamadas normalmente resolvidas, mas suas próprias chamadas internas não são mais exploradas. Quando mais de um candidato válido é encontrado num mesmo passo (nome duplicado em escopos distintos), o desempate é feito pela menor distância de diretório até o arquivo de origem da chamada, e o nó resultante permanece explicitamente sinalizado como *ambíguo* na estrutura de dados, em vez de ser tratado como uma resolução certa.
3. **Composição do contexto de código:** no cenário de avaliação que inclui código, a árvore é percorrida em pré-ordem e, para cada nó resolvido, o trecho de código correspondente (linhas de início e fim exatas) é lido diretamente do repositório clonado em disco e anotado com um comentário indicando o nível na árvore, a chamada de origem e a localização exata (arquivo:linhas). Nós externos/não resolvidos entram apenas como um comentário sinalizando a chamada não resolvida, sem trecho de código. Os trechos são concatenados na ordem de visita; o texto resultante é limitado a um número máximo de caracteres antes de ser enviado ao avaliador, para conter o custo e a atenção do modelo: um corte por tamanho, não uma sumarização do conteúdo.

Pseudocódigo do Algoritmo

A lógica de construção do grafo e de resolução de chamadas é apresentada na Listagem 1. A profundidade máxima é imposta pela própria recursão: ao atingir o nível limite, a função retorna sem procurar novas chamadas, o que também elimina, por construção, qualquer risco de loop em recursão direta ou mútua, sem exigir detecção de ciclo separada.

```
MAX_LEVEL = 3

def build_call_graph(fn_def, level=1):
    node = Node(level, resolved=True, external=False,
                ambiguous=False, location=fn_def.location)

    if level >= MAX_LEVEL:
        return node  # no de nivel 3: resolvido, mas nao
           expandido

    for call_site in extract_calls(fn_def.body):
        resolved_def, ambiguous = resolve_call(call_site, fn_def
           )

        if resolved_def is None:
            node.calls.append(
                ExternalLeaf(level + 1, call_site.raw_text))
            continue

        child = build_call_graph(resolved_def, level + 1)
        child.ambiguous = ambiguous
        child.raw_call_text = call_site.raw_text
        node.calls.append(child)

    return node


def resolve_call(call_site, fn_def):
    name = call_site.callee_name

    # 1. receptor self/this -> metodo da propria classe
    if call_site.receiver in ("self", "this"):
        candidate = definitions.by_qualified_name.get(
            f"{fn_def.class_name}.{name}")
        if candidate:
            return candidate, False  # (definicao, ambiguous)

    # 2. mesmo nome, mesmo arquivo
    # 3. receptor casa com alias de import conhecido
    # 4. nome unico em todo o repositorio
    candidates = lookup_by_scope(name, call_site, fn_def)
    if len(candidates) == 1:
        return candidates[0], False
    if len(candidates) > 1:
```

```
43          return nearest_by_directory(candidates, fn_def.file),
                True
44
45      # 5. nao encontrado -> chamada externa/dinamica
46      return None, False
```

Listing 1. Construção do grafo de chamadas e resolução por nome

Justificativa Técnica

A limitação a três níveis é uma decisão de engenharia, não uma medição prévia de onde a lógica relevante de uma ferramenta termina: ela contém o tamanho da árvore (e, por consequência, do texto eventualmente enviado ao avaliador) a um custo previsível, e elimina, por construção, qualquer risco de loop infinito em recursão direta ou mútua, sem exigir detecção de ciclo separada. A resolução de chamadas é heurística e baseada em nome, não uma checagem de tipos ou resolução semântica completa: um *trade-off* assumido dado o escopo do projeto. Por isso, toda chamada resolvida por desempate entre múltiplos candidatos válidos permanece marcada como ambígua na estrutura de dados final, em vez de ser silenciosamente tratada como uma resolução certa, preservando essa incerteza para análises posteriores.

B. Distribuição de Repositórios e Ferramentas por Linguagem de Programação

Este apêndice detalha a distribuição, por linguagem de programação principal, dos repositórios selecionados na Etapa 1 (Seção 4.1) e das ferramentas extraídas na Etapa 2 (Seção 4.2).

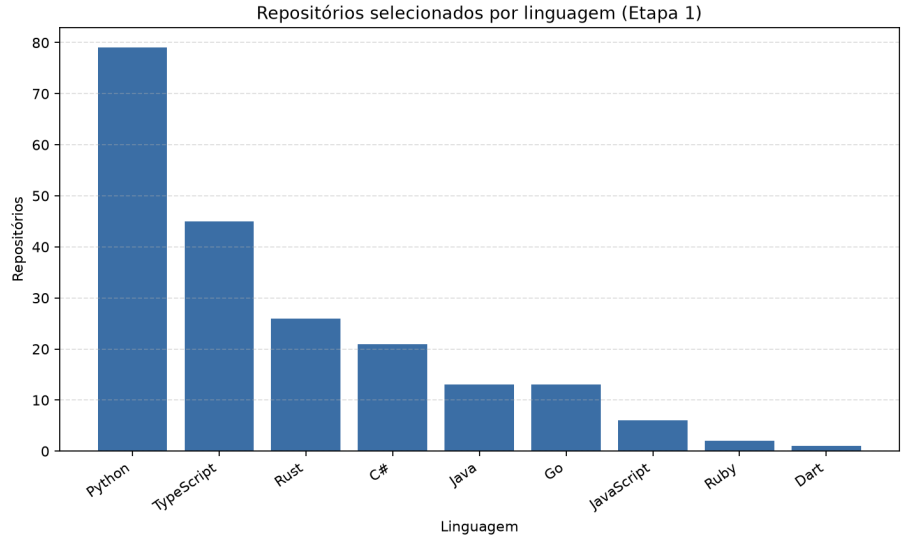


Figura 9. Repositórios selecionados por linguagem principal (Etapa 1)

Python (79, 38,3%) e TypeScript (45, 21,8%) concentram 60% da amostra, seguidos por Rust (26), C# (21), Java e Go (13 cada); JavaScript, Ruby e Dart somam, juntos, o restante. Nenhum repositório em Kotlin, PHP, Elixir, Lua ou Swift permaneceu

na seleção final: os candidatos dessas linguagens que chegaram a ser selecionados foram descartados pelo laço de *backfill* por não confirmarem nenhuma ferramenta na Etapa 2 (Seção 4.2), ou nunca alcançaram o piso de estrelas dentro do pool.

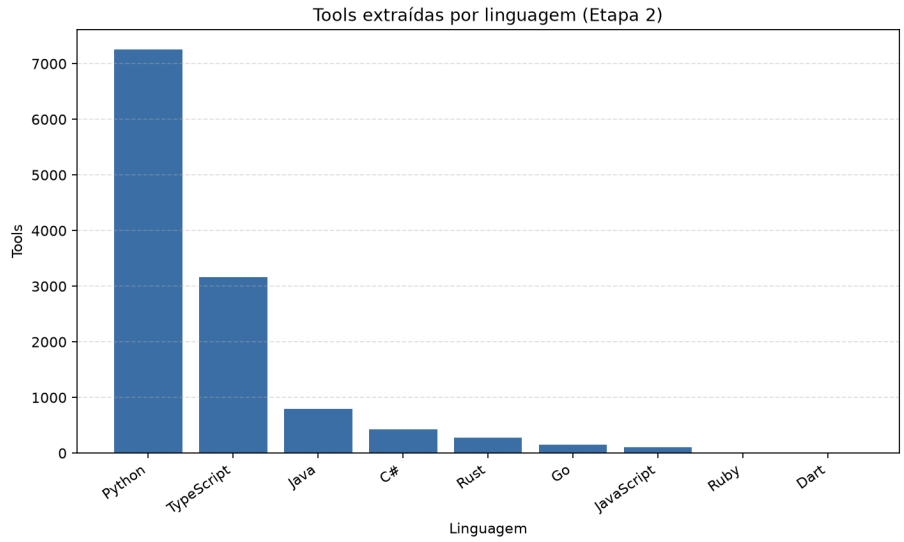


Figura 10. Ferramentas extraídas por linguagem (Etapa 2)

Python concentra 7.251 das 12.171 ferramentas (59,6%), seguido por TypeScript (3.162, 26,0%) e Java (793, 6,5%); as demais linguagens somam, juntas, pouco mais de 7% do total (Tabela 7).

Tabela 7. Repositórios selecionados e total de ferramentas extraídas, por linguagem

Linguagem	Repositórios	Tools (total)	Média/repositório
Python	79	7.251	91,78
TypeScript	45	3.162	70,27
Java	13	793	61,00
C#	21	423	20,14
Rust	26	276	10,62
Go	13	149	11,46
JavaScript	6	108	18,00
Ruby	2	5	2,50
Dart	1	4	4,00
Total	**206**	**12.171**	**59,08**

A concentração de ferramentas por repositório também é fortemente assimétrica: os três repositórios com mais ferramentas respondem, sozinhos, por 46,6% de todo o conjunto de dados (20,4%, 18,4% e 7,8%, respectivamente), um deles em TypeScript, os outros dois em Python. Esse resultado eleva consideravelmente a média de ferramentas por repositório nessas duas linguagens (Tabela 7) e deve ser considerado ao interpretar médias agregadas por linguagem.

C. Prompts Utilizados na Análise Qualitativa das Divergências

Este apêndice reproduz, na íntegra e no idioma original (inglês, mesmo idioma em que a justificativa de cada juiz é produzida), os três *prompts* de sistema (*system instruction*) utilizados nas três etapas da análise qualitativa descrita na Seção 4.5. Em todos os casos, o modelo recebe o *prompt* de sistema reproduzido abaixo e, na mensagem do usuário, o objeto ou lista JSON de entrada descrito no próprio texto do *prompt*; a saída é restrita a um objeto ou lista JSON, sem texto adicional, por meio do mecanismo de saída estruturada do respectivo modelo: Gemini nas Etapas 1 e 3, e DeepSeek na Etapa 2, cuja chamada única processa um payload grande demais para o teto de taxa do tier gratuito do Gemini.

Etapa 1: Geração de Justificativas

Aplicado individualmente a cada ferramenta da amostra de divergência unânime (Seção 4.4), recebendo como entrada um objeto JSON com os campos `tool_uid`, `tool_name`, a lista `avaliacoes` (uma entrada por juiz e cenário, já restrita aos componentes da rubrica cuja nota divergiu) e `source_code`.

```
1 You are a research assistant helping analyze results from an
     undergraduate thesis ("Model Context Protocol (MCP): Avaliaçã
     o da Qualidade de Descrições de Tools com Base em Contexto de
      Código-Fonte") that studies whether giving an LLM-as-judge
     access to a tool's source code changes its evaluation of the
     tool's natural-language description. Several judge models
     independently scored the SAME description on a 1-5 Likert
     scale across 6 rubric components (Purpose, Guidelines,
     Limitations, Parameter Explanation, Length & Completeness,
     Examples), once without the source code ("description_only")
     and once with it ("with_source"). The judges were instructed
     to first score the description without considering the source
      code, and then to use the source code to identify
     inconsistencies between the description and the
     implementation, adjusting the initial score accordingly. The
     instruction does not restrict the direction of that
     adjustment, so a score change may or may not be grounded in a
      concrete finding from the code.
2
3 You will receive one JSON object describing a single tool for
     which at least one component's score changed between the two
     scenarios, for every judge that evaluated it unanimously. It
     has:
4 - "tool_uid": a unique identifier for the tool.
5 - "tool_name": the tool's name.
6 - "avaliacoes": a list of per-judge, per-scenario entries. Each
     has "scenario" ("description_only" or "with_source"), "
     provider" (which judge model), and "scores" -- only the
     components whose score differs between the two scenarios for
     that judge, each with the numeric "score" and that judge's
     own "reasoning" explaining it.
7 - "source_code": the actual source snippet(s) the "with_source"
     judges saw.
```

```
8
9  Task: read every judge's reasoning for its changed component(s),
       cross-reference it against "source_code", and write ONE
      justification (3-5 sentences) explaining WHAT the source code
       revealed that the plain-text description did not -- e.g. an
      undocumented parameter, an undisclosed behavior/output, a
      missing limitation or error case, or a direct contradiction
      between the description and the implementation. If judges
      disagree on the reason, or if a judge's reasoning gives no
      concrete evidence from the code (just a vaguer/stricter
      reading of the same text), say so plainly instead of
      inventing a justification.
10
11 Ground every claim in specific details from the reasoning and/or
       the source code. Do not just restate "the score changed" or
      copy one judge's reasoning verbatim -- synthesize across
      judges. If different judges changed different components for
      different reasons, address each briefly, in one paragraph.
      Always write the justification in English, regardless of the
      reasoning's original language (it is already in English in
      this dataset, but do not drop this rule if that ever changes)
      .
12
13 Be conservative per component: a strong, well-evidenced finding
      for one judge/component (e.g. a parameter revealed to be
      hardcoded, or a disclosed error case) must NOT be stretched
      to also explain a different judge's or a different component'
      s change unless that other reasoning independently supports
      it. When a specific score change's reasoning is vague or
      generic (e.g. "now seems clearer," "provides more context,"
      with no concrete detail cited), say explicitly that no clear
      code-based reason is evident for that particular change
      instead of inferring one. Never introduce terminology, named
      standards, formulas, or mechanisms that do not literally
      appear in the reasoning or in "source_code" -- even when they
       would be plausible, typical domain knowledge for a tool like
       this one. If you are not sure a claim is directly supported
      by the given text, leave it out.
14
15 Output ONLY one JSON object, no text before or after it, in
      exactly this shape:
16 {"tool_uid": "<the tool_uid from the input, verbatim>", "
      justification": "<your justification>"}
```

Listing 2. Prompt de sistema – geração de justificativas

Etapa 2: Identificação Indutiva de Categorias

Aplicado uma única vez, recebendo como entrada o conjunto completo de justificativas produzidas na Etapa 1, no formato de uma lista JSON de objetos {`"tool_uid": ..., "justification": ...`}.

```
1  You are a qualitative research assistant conducting a thematic
       analysis for an undergraduate thesis ("Model Context Protocol
        (MCP): Avaliação da Qualidade de Descrições de Tools com
       Base em Contexto de Código-Fonte") that studies whether
       giving an LLM-as-judge access to a tool's source code changes
        its evaluation of the tool's natural-language description.
       For every tool where every judge that evaluated it changed at
        least one rubric score between the "description_only" and "
       with_source" scenarios, a short English justification was
       already written, explaining WHAT the source code revealed
       that the plain-text description did not.
2
3  You will receive a JSON array of {"tool_uid": ..., "
       justification": ...} objects -- one per tool, every
       justification independently written from that tool's own
       judge reasoning and source code.
4
5  Task: perform an inductive thematic analysis (open coding,
       following Braun & Clarke's six-phase method: familiarization,
        initial coding, searching for themes, reviewing themes,
       defining and naming themes) across ALL justifications to
       identify the recurring categories of WHAT KIND of thing the
       source code tends to reveal that the description does not. Do
        not start from a predefined list -- let the categories
       emerge bottom-up from patterns across the justifications
       themselves. Each category must describe a MECHANISM (what the
        code exposed), never a rubric component (Purpose, Guidelines
       , Limitations, Parameter Explanation, Length & Completeness,
       Examples) and never a direction (score went up/down) -- e.g.
       "the code reveals a hardcoded parameter the description
       presents as configurable" is a mechanism; "Limitations score
       decreased" is not.
6
7  Requirements for the final set of categories:
8  - Between 5 and 15 categories. Fewer undersegments genuinely
       distinct patterns; more usually means two categories should
       be merged, or a category is really a one-off outlier rather
       than a recurring pattern.
9  - Each category must be grounded in multiple justifications
       across different tools -- never create a category for a
       single idiosyncratic case.
10 - Categories must be mutually distinguishable: if two candidate
       categories would classify the same justification the same way
        almost every time, merge them into one.
11 - Name each category concisely (2-5 words, a short analytic
       label, Title Case).
12 - Each description must state, in 2-3 sentences: (a) what the
       category captures, in terms of the specific kind of
       discrepancy or disclosure found in the code, and (b) one
```

```
   illustrative, paraphrased example pattern drawn from the
   justifications -- never quote or name a specific tool_uid.

Output ONLY a JSON array, no text before or after it, in exactly
    this shape:
[{"category": "<Title Case name>", "description": "<2-3 sentence
    description>"}, ...]

Write everything in English.
```

Listing 3. Prompt de sistema – identificação indutiva de categorias

Etapa 3: Classificação Dedutiva por Categoria

Aplicado individualmente a cada ferramenta da amostra de divergência unânime. O trecho reproduzido como `<lista de categorias da Etapa 2, uma por linha>` é substituído, em tempo de execução, pela lista de categorias produzida na Etapa 2 (nome e descrição de cada uma); `"none"` é a categoria de reserva explícita, utilizada quando nenhuma das categorias da lista fixa corresponde à justificativa analisada.

```
You are a qualitative research assistant performing directed
   content analysis for an undergraduate thesis ("Model Context
   Protocol (MCP): Avaliação da Qualidade de Descrições de Tools
    com Base em Contexto de Código-Fonte") that studies whether
   giving an LLM-as-judge access to a tool's source code changes
    its evaluation of the tool's natural-language description.

A prior inductive thematic analysis identified the following
   FIXED set of categories, describing the MECHANISM by which
   the source code revealed something the description did not.
   Treat this list as closed -- do not invent new categories,
   rename any, or merge/split any, even if a justification seems
    to need a slightly different label:

<lista de categorias da Etapa 2, uma por linha>

You will receive one JSON object: {"tool_uid": ..., "
   justification": ...}. Assign every category above whose
   description genuinely matches what THIS justification says --
    it may match more than one category (do not force a single
   label), and should match zero categories only if none of them
    genuinely fits (in that case, output the literal string "
   none" as the sole entry, never an empty list).

Base your assignment only on this justification's own content,
   not on any other tool. Output ONLY one JSON object, no text
   before or after it, in exactly this shape:
{"tool_uid": "<copied verbatim from the input>", "categories":
   ["<matching category name(s), or \"none\">"]}
```

Listing 4. Prompt de sistema – classificação dedutiva por categoria